```
-- It should hang since the table is locked for exclusive access
-- But it should get canceled after 5s
ALTER TABLE trips ADD COLUMN city_id INTEGER;

-- In psql2 notice the statement is canceled
-- ERROR:  canceling statement due to lock timeout

-- run rollback in both psql1 and psql2
ROLLBACK;
```

You've now seen how to set a max allowable time for lock acquisition. What other kinds of safeguards can be added?

Exploring Lock Type Queues

Statements requesting access for a lock type are put into a queue where they wait in order.

If transaction B is waiting to acquire the lock type that transaction A has exclusive access to, B must wait for A.

As described in *PostgreSQL and the Lock Queue*,[7] the pids for the backend processes are logged including the statement holding a lock type and the statements waiting to acquire it.

Some helpful lock parameters are to set log_lock_waits to on and setting deadlock_timeout to get more visibility into locks or blocked queries. When these timeouts cause cancellations they'll be logged to the postgresql.log. See the post "Cost of log_lock_waits' and deadlock_timeout'."[8]

Take a look at the following example. This example shows the pid holding a lock and the pids waiting for the lock type.

```
DETAIL: Processes holding the lock: 29386. Wait queue: 31025, 30551, ...
```

The pids 31025 and 30551 are blocked waiting on pid 29386. Once the query associated to the backend pid 29386 completes, the pics that are waiting will be processed in the order of how they appear.

Besides using the pg_stat_activity system catalog you learned about earlier to view lock related queries, you may want to use a log analyzer that analyzes your postgresql.log file.

The log analyzer program pgBadger[9] organizes lock related query information it parses from your log file. The lock information is put into categories like

7. https://joinhandshake.com/blog/our-team/postgresql-and-lock-queue/
8. https://dba.stackexchange.com/a/249904
9. https://github.com/darold/pgbadger